



Morpho: Midnight Security Review

Auditors

MiloTruck, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

Jonatas Martins, Security Researcher

Report prepared by: Lucas Goiriz

August 8, 2026

Contents

1	About Spearbit	2
2	Introduction	2
3	Risk classification	2
3.1	Impact	2
3.2	Likelihood	2
3.3	Action required for severity levels	2
4	Executive Summary	3
4.1	Scope	3
5	Findings	4
5.1	Medium Risk	4
5.1.1	Zero Oracle Prices are allowed	4
5.1.2	Obligation can be created before default fees are set for a loan token	4
5.2	Low Risk	5
5.2.1	Deltas of pending fee for buyer, seller and position updates round in the wrong direction	5
5.2.2	Input validation is missing for <code>offer.obligation.maturity</code>	6
5.2.3	Enforce stricter trading fee rounding	7
5.2.4	During liquidation explicit check is missing to ensure <code>collateralIndex</code> is one the activated collaterals	9
5.2.5	When <code>lossIndex</code> is at its max withdrawal would not be possible but other endpoints can be called	10
5.2.6	Foreign ratifiers can import delegated signer authority from another Midnight instance	10
5.2.7	<code>ApprovalRatifier</code> approvals can replay across Midnight deployments	11
5.2.8	Calls to <code>take()</code> can be forced to revert via front-running	12
5.2.9	Continuous fee calculation could round down to zero for tokens with small decimals	12
5.2.10	Obligation accounting breaks in the event of a hardfork	13
5.3	Gas Optimization	13
5.3.1	Verification and potential optimisation of <code>countBits</code> implementation	13
5.4	Informational	17
5.4.1	Minor issues (typos, comments, ...)	17
5.4.2	A phantom collateral can block liquidations	18
5.4.3	In the presence of liquidity the lenders and the <code>feeClaimer</code> can avoid their credits getting slashed	19
5.4.4	<code>clz</code> opcode is not supported on all almost EVM-compatible chains	19
5.4.5	Trust assumptions for <code>buyerCallback</code>	21
5.4.6	Loss index inflation	21
6	Appendix	23
6.1	Formulas and invariants	23
6.1.1	Credit/Debt	23
6.1.2	Some invariants	23
6.1.3	Notation	23
6.2	Debt Accounting + Liquidation	24
6.3	Risky collaterals	26
6.3.1	Notations	26

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Morpho is a trustless and efficient lending primitive with permissionless market creation.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho: Midnight according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 4 days in total, [Morpho](#) engaged with [Spearbit](#) to review the [midnight](#) protocol. In this period of time a total of **19** issues were found.

Summary

Project Name	Morpho
Repository	midnight
Commit	41c8405a
Type of Project	DeFi, Lending
Audit Timeline	Apr 7th to Apr 21st

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	0	2
Low Risk	10	6	4
Gas Optimizations	1	0	1
Informational	6	4	2
Total	19	10	9

4.1 Scope

The security review had the following components in scope for [midnight](#) on commit hash [41c8405a](#):

```
src
├── Midnight.sol
├── authorizers
│   └── EcrecoverAuthorizer.sol
├── libraries
│   ├── ConstantsLib.sol
│   ├── EventsLib.sol
│   ├── IdLib.sol
│   ├── SafeTransferLib.sol
│   ├── TickLib.sol
│   └── UtilsLib.sol
├── periphery
│   └── TakeAmountsLib.sol
└── ratifiers
    ├── ApprovalRatifier.sol
    └── EcrecoverRatifier.sol
```

5 Findings

5.1 Medium Risk

5.1.1 Zero Oracle Prices are allowed

Severity: Medium Risk.

Context: [Midnight.sol#L531-L532](#).

Description: If for some reason an oracle would return `0` for a collateral price. A non-healthy or matured position (with debt) can be liquidated and all its collateral corresponding to this oracle can be seized without re-paying any loan tokens:

```
if (seizedAssets > 0) {
    repaidUnits = seizedAssets.mulDivUp(liquidatedCollatPrice,
    ↪ ORACLE_PRICE_SCALE).mulDivUp(WAD, lif);
} else { ... }
```

Recommendation: The above case perhaps should not be allowed and to be carefully documented. The Nat-Spec currently has the following clause in [Midnight.sol#L88-L89](#):

```
/// @dev If an activated collateral oracle returns 0 on `price`, `isHealthy`,
↪ `withdrawCollateral` when the borrower has
/// debt, `take` whenever the seller still has debt, and `liquidate` with repaid input all
↪ revert.
```

which does not cover the above scenario.

It would be best to check that if a provided input to the liquidation has `seizedAssets>0` then `repaidUnits` should also be non-zero.

Morpho: We acknowledge this issue, as we don't think that there is something misleading.

Spearbit: Acknowledged.

5.1.2 Obligation can be created before default fees are set for a loan token

Severity: Medium Risk.

Context: (No context files were provided by the reviewer).

Description: In `touchObligation()`, fees are taken from the default values and stored in `obligationState` whenever an obligation is created:

```
uint16[7] memory _defaultTradingFees = defaultTradingFees[obligation.loanToken];
_obligationState.fee0 = _defaultTradingFees[0];
_obligationState.fee1 = _defaultTradingFees[1];
_obligationState.fee2 = _defaultTradingFees[2];
_obligationState.fee3 = _defaultTradingFees[3];
_obligationState.fee4 = _defaultTradingFees[4];
_obligationState.fee5 = _defaultTradingFees[5];
_obligationState.fee6 = _defaultTradingFees[6];
_obligationState.continuousFee = defaultContinuousFee[obligation.loanToken];
```

However, a user could call `touchObligation()` even before the default fees are set to effectively have no fees.

This is applicable for assets which could be used as `loanToken` at a later time, since it's not possible to set the default fee for them immediately upon deployment (eg. a token deployed in the future).

The `feeSetter` can correct this by calling `setObligationTradingFee()` / `setObligationContinuousFee()` for the specific obligation, but there is still a window to take advantage of this. For example:

1. A new token is deployed.
2. A taker matches a maker and calls `take()` (and `touchObligation()` indirectly) immediately after.
3. The fee setter sets the default fees and corrects the obligation created in (2).

However, the taker/maker in (2) did not incur any trading fee, and will not have any continuous fee as the pending fee is zero at time of calling.

Recommendation: A potential mitigation would be to introduce a global default trading fee, instead of a default per loan token.

Morpho: We acknowledge this behavior (was known). And we don't think that it should be documented, as it's not misleading nor hidden.

Spearbit: Acknowledged.

5.2 Low Risk

5.2.1 Deltas of pending fee for buyer, seller and position updates round in the wrong direction

Severity: Low Risk.

Context: [Midnight.sol#L330-L334](#), [Midnight.sol#L418-L419](#), [Midnight.sol#L711](#).

Description:

- `take` : the contract computes the buyer's newly inherited pending fee with `mulDivDown` and the seller's released pending fee with `mulDivUp` :

```
uint128 buyerPendingFeeIncrease =
    UtilsLib.toUint128(buyerCreditIncrease.mulDivDown(_obligationState.continuousFee *
        ↪ timeToMaturity, WAD));
uint128 sellerPendingFeeDecrease = sellerPos.credit > 0
    ? UtilsLib.toUint128(sellerPos.pendingFee.mulDivUp(sellerCreditDecrease, sellerPos.credit))
    : 0;
```

- `updatePositionView` : post slash pending fee uses `mulDivUp` .

```
uint256 postSlashPending = credit > 0 ? _pendingFee - _pendingFee.mulDivUp(credit -
    ↪ postSlashCredit, credit) : 0;
```

- `withdraw` : pending fee decrease uses `mulDivUp` :

```
pendingFeeDecrease = UtilsLib.toUint128(_position.pendingFee.mulDivUp(units,
    ↪ _position.credit));
```

These rounding directions are reversed.

Recommendation: Reverse the rounding directions:

- Round `buyerPendingFeeIncrease` up, and.
- Round `sellerPendingFeeDecrease` down.
- Round `postSlashPending` up.
- Round `pendingFeeDecrease` down.

That is:

```
uint128 buyerPendingFeeIncrease =
    UtilsLib.toUint128(buyerCreditIncrease.mulDivUp(_obligationState.continuousFee *
        ↪ timeToMaturity, WAD));
uint128 sellerPendingFeeDecrease = sellerPos.credit > 0
    ? UtilsLib.toUint128(sellerPos.pendingFee.mulDivDown(sellerCreditDecrease,
        ↪ sellerPos.credit))
    : 0;
```

and.

```
uint256 postSlashPending = credit > 0 ? _pendingFee - _pendingFee.mulDivDown(credit -
    ↪ postSlashCredit, credit) : 0;
```

and.

```
pendingFeeDecrease = UtilsLib.toUint128(_position.pendingFee.mulDivDown(units,
    ↪ _position.credit));
```

This makes fee accounting conservative in the protocol's favor and also follows the same rounding direction pattern in.

Morpho: Acknowledged.

Spearbit Acknowledged.

5.2.2 Input validation is missing for `offer.obligation.maturity`

Severity: Low Risk.

Context: [Midnight.sol#L330-L331](#).

Description: Upon touching an offer's obligation the 1st time, no input validation gets performed for the `maturity` field which has a type `uint256`. Given a very high value for this parameter the delta pending fee calculated for the buyer can be greater than the corresponding delta credit for the buyer. This would break an invariant which one might assume $f \leq a$.

Moreover, expired obligations can still be instantiated as dead markets ([Midnight.sol#L658](#), [Midnight.sol#L447](#), [Midnight.sol#L403](#), [Midnight.sol#L835](#)). `touchObligation` accepts a maturity already in the past, so the first interaction can create an obligation that is already expired and can even accept collateral. There doesn't seem to be a solvency break from this, because any fresh `take` that would create seller debt reverts at the final liquidatability check, and `liquidate` still requires pre-existing debt. The issue is that `ObligationCreated` no longer implies a live borrowable market, which is a footgun for integrators and users. If this state is not intentional, reject `maturity < block.timestamp` on creation; otherwise document that created may still mean exit-only/dead on arrival.

Recommendation:

1. It might make sense to limit the given `maturity` by perhaps a value like 100 years from the timestamp the obligation gets touched/created (one can see this is safe when combined with the `MAX_CONTINUOUS_FEE` value).

Otherwise, it would be best to document this to let buyer know so they can avoid obligations with very long maturity dates. `feeSetter` could set `_obligationState.continuousFee` to `0` for these types of obligations.

2. Disallow creation of dead markets or document that creation of markets with past maturity is allowed.

Overall, one is hoping for checks of the form:

$$t_{creation} \leq t_m \leq t_{creation} + t_{fixed}$$

Also make sure the test suite covers the above scenarios.

Morpho: Fixed partially in [PR 742](#).

Spearbit: The first recommended fix was implemented in [PR 742](#). The second point was acknowledged.

5.2.3 Enforce stricter trading fee rounding

Severity: Low Risk.

Context: [Midnight.sol#L301-L307](#), [Midnight.sol#L391-L392](#).

Description: One can show that the value of `buyerAssets-sellerAssets` lies in the set:

$$\left\{ \left\lfloor \frac{f_{trade}}{10^{18}} a \right\rfloor, \left\lceil \frac{f_{trade}}{10^{18}} a \right\rceil \right\}$$

i.e.

```
_tradingFee.mulDivDown(units, WAD) // or
_tradingFee.mulDivUp (units, WAD) // or
```

so there could be scenarios where where for a provided `unit a` one would end up paying no trading fees (`buyerAssets-sellerAssets==0`) even though `_tradingFee` could be non-zero.

Recommendation:

Recommendation 1: To enforce that trading fees are always paid to `Midnight` when `unit` and `_tradingFee` are non-zero, it might make sense to apply the following change:

```
diff --git a/src/Midnight.sol b/src/Midnight.sol
index 018f8904..bc12252c 100644
- -- a/src/Midnight.sol
+ ++ b/src/Midnight.sol
@@ -302,9 +302,8 @@ contract Midnight is IMidnight {
    uint256 timeToMaturity = UtilsLib.zeroFloorSub(offer.obligation.maturity,
    ↪ block.timestamp);
    uint256 _tradingFee = tradingFee(id, timeToMaturity);
    uint256 sellerPrice = offer.buy ? offerPrice - _tradingFee : offerPrice;
-   uint256 buyerPrice = sellerPrice + _tradingFee;
-   uint256 buyerAssets = offer.buy ? units.mulDivDown(buyerPrice, WAD) :
↪ units.mulDivUp(buyerPrice, WAD);
    uint256 sellerAssets = offer.buy ? units.mulDivDown(sellerPrice, WAD) :
    ↪ units.mulDivUp(sellerPrice, WAD);
+   uint256 buyerAssets = sellerAssets + units.mulDivUp(_tradingFee, WAD);

    uint256 newConsumed;
    if (offer.maxSellerAssets > 0) {
```

This would make sure that:

$$a_{buyer} - a_{seller} = \left\lceil \frac{f_{trade}}{10^{18}} a \right\rceil$$

There is a slight caveat where when the `maker` is the `buyer` it might spend slightly more than expected (at most by one wei of the loan token):

$$\left\lfloor \frac{p(i)}{10^{18}} a \right\rfloor \leq a_{buyer} \leq \left\lfloor \frac{p(i)}{10^{18}} a \right\rfloor + 1$$

since.

$$a_{limit} = \frac{p(i)}{10^{18}} a < \left\lfloor \frac{p(i)}{10^{18}} a \right\rfloor + 1$$

Recommendation 2: To fix the above caveat, one can instead apply the following changes:

```
diff --git a/src/Midnight.sol b/src/Midnight.sol
index 018f8904..6de58c20 100644
- -- a/src/Midnight.sol
+ ++ b/src/Midnight.sol
@@ -301,10 +301,11 @@ contract Midnight is IMidnight {
    uint256 offerPrice = TickLib.tickToPrice(offer.tick);
    uint256 timeToMaturity = UtilsLib.zeroFloorSub(offer.obligation.maturity,
    ↪ block.timestamp);
    uint256 _tradingFee = tradingFee(id, timeToMaturity);
-    uint256 sellerPrice = offer.buy ? offerPrice - _tradingFee : offerPrice;
-    uint256 buyerPrice = sellerPrice + _tradingFee;
-    uint256 buyerAssets = offer.buy ? units.mulDivDown(buyerPrice, WAD) :
    ↪ units.mulDivUp(buyerPrice, WAD);
-    uint256 sellerAssets = offer.buy ? units.mulDivDown(sellerPrice, WAD) :
    ↪ units.mulDivUp(sellerPrice, WAD);
+
+    uint256 feeAssets = units.mulDivUp(_tradingFee, WAD);
+    uint256 offerAssets = offer.buy ? units.mulDivDown(offerPrice, WAD) :
    ↪ units.mulDivUp(offerPrice, WAD);
+    uint256 sellerAssets = offer.buy ? offerAssets - feeAssets : offerAssets;
+    uint256 buyerAssets = sellerAssets + feeAssets;

    uint256 newConsumed;
    if (offer.maxSellerAssets > 0) {
@@ -388,8 +389,8 @@ contract Midnight is IMidnight {
    }

    address payer = buyerCallback != address(0) ? buyerCallback : (offer.buy ? buyer :
    ↪ msg.sender);
-    SafeTransferLib.safeTransferFrom(offer.obligation.loanToken, payer, address(this),
    ↪ buyerAssets - sellerAssets);
-    claimableTradingFee[offer.obligation.loanToken] += buyerAssets - sellerAssets;
+    SafeTransferLib.safeTransferFrom(offer.obligation.loanToken, payer, address(this),
    ↪ feeAssets);
+    claimableTradingFee[offer.obligation.loanToken] += feeAssets;
    SafeTransferLib.safeTransferFrom(offer.obligation.loanToken, payer, receiver,
    ↪ sellerAssets);

    if (sellerCallback != address(0)) {
```

then the prices and limits asset bounds would be guaranteed for the `maker`.

- `maker` is the `buyer` (for this case there is a possibility that `offerAssets-feeAssets` might cause a revert. But this is also related to the current implementation where potentially `offerPrice-_tradingFee` could revert):

$$a_{buyer} = \left\lfloor \frac{p(i)}{10^{18}} a \right\rfloor \leq \frac{p(i)}{10^{18}} a = a_{limit}$$

$$a_{seller} = \left\lfloor \frac{p(i)}{10^{18}} a \right\rfloor - \left\lceil \frac{f_{trade}}{10^{18}} a \right\rceil$$

- `maker` is the `seller` :

$$a_{buyer} = \left\lfloor \frac{p(i)}{10^{18}} a \right\rfloor + \left\lceil \frac{f_{trade}}{10^{18}} a \right\rceil$$

$$a_{seller} = \left\lfloor \frac{p(i)}{10^{18}} a \right\rfloor \geq \frac{p(i)}{10^{18}} a = a_{limit}$$

This design has the desired property that splitting a `take` call would incur more fees:

$$\left\lceil \frac{f_{trade}}{10^{18}} \left(\sum_{i=0}^n a_i \right) \right\rceil \leq \sum_{i=0}^n \left\lceil \frac{f_{trade}}{10^{18}} a_i \right\rceil$$

Morpho: We decided to acknowledge this.

Spearbit: Acknowledged.

5.2.4 During liquidation explicit check is missing to ensure `collateralIndex` is one the activated collaterals

Severity: Low Risk.

Context: [Midnight.sol#L528-L539](#), [Midnight.sol#L576](#), [Midnight.sol#L595](#).

Description: During liquidation explicit check is missing to ensure `collateralIndex` is one the activated collaterals. There are 3 possible cases:

Case 1. Almost no-op (only bad debt accounting): In this case `repaidUnits==seizedAssets==0` and potentially bad debt accounting gets performed. The provided `collateralIndex` can be outside of the activated collateral sets of the `borrower` in the obligation. This value gets consumed by.

- The event `EventsLib.Liquidate` .
- The zero collateral token transfer.
- The potential callback to the liquidator.

Case 2. `repaidUnits > 0` : In this case the call would revert if `collateralIndex` is not in the activated collateral set of the borrower since `liquidatedCollatPrice` ends up being `0` and gets used in the denominator to calculate `seizedAssets` .

Case 3. `seizedAssets > 0` : This case should also revert due to how `newCollateral` (if one assumes the invariants of the `_position.activatedCollaterals` are not broken):

```
uint128 newCollateral = _position.collateral[collateralIndex] -
↳ UtilsLib.toUint128(seizedAssets);
```

Since in this case `_position.collateral[collateralIndex]` should be `0` and one cannot subtract a positive value of `seizedAssets` from it.

Recommendation: One can indirectly see that the missing validation does not affect the last cases **2.** and **3.** as the call would be reverted. But it does affect case **1.** It is recommended to perform a more explicit check of the following form:

```
require(bitmap.isBitSet(collateralIndex), "collateral index is not activated");
while (bitmap != 0) { ... }
```

(define the `isBitSet` utility function accordingly.).

Morpho: We decided that it wasn't worth fixing. You would rather want to see that realizing the bad debt is always possible. And the "weird" events are manageable. We added a comment in [PR 785](#).

Spearbit: Fixed by commenting it.

5.2.5 When `lossIndex` is at its max withdrawal would not be possible but other endpoints can be called

Severity: Low Risk.

Context: [Midnight.sol#L408-L413](#).

Description: When `lossIndex==type(uint128).max`, one would not be able to withdraw any of its credits anymore since the call to `_updatePosition(...)` sets the position's credit to `0` (if not already being at `0`). Thus any old or new credits would be perceived as `0`. So the call to `withdraw` with any portions `units` amount would revert. For the same obligation at this state, one can still call:

- `take` to buy credits (which will later be valued at `0`) and accrue debts (for the seller).
- `repay` to reduce its debt. This might make sense since one would want to withdraw its collateral and to see it still needs to keep a healthy position. But the repaid `units` would not be able to be used by anyone and forever locked in the contract.
- `supplyCollateral` and `withdrawCollateral` (would make sense to allow).
- `liquidation`. The effect on `lossIndex` would be idempotent. The transferred `repaidUnits` loan tokens would not be able to be used by anyone (forever locked in the contract).

Recommendation: In this potentially rare scenario:

- Calling `take` should be blocked.
- Calling `repay` and `liquidate` to repay loan tokens might not make sense and a different mechanism like unrolling credits for the lenders might be beneficial.

Morpho: Fixed in [PR 743](#).

Spearbit: Fix verified.

5.2.6 Foreign ratifiers can import delegated signer authority from another Midnight instance

Severity: Low Risk.

Context: [EcrecoverRatifier.sol#L17-L26](#).

Description: `Midnight.take` only checks that the maker authorized `offer.ratifier` in the current Midnight instance before delegating trust to the external ratifier. `EcrecoverRatifier.onRatify`, however, validates the recovered signer against its own immutable `MIDNIGHT` address instead of the caller.

As a result, if a maker authorizes the same ratifier R on multiple Midnight deployments M_A, M_B a signer who is authorized for that maker on M_A can ratify an order that executes on M_B .

This crosses an otherwise natural trust boundary: delegated signer permissions are expected to be local to each Midnight deployment, but the ratifier imports them from a different instance.

This is made easier by the fact that `Offer` (`IMidnight.sol#L21-L38`) and the signed `Root` (`IEcrecover.sol#L21-L22`) do not bind the current Midnight address. In practice this is primarily a same-chain multi-deployment issue. Cross-chain replay is normally blocked by the EIP-712 `chainId`, but the boundary still fails if environments intentionally reuse the same `chainid` and ratifier address layout.

Recommendation: Ratifiers should be bound to the Midnight instance that invokes them. At minimum, `EcrecoverRatifier` should reject foreign callers with `require(msg.sender==MIDNIGHT)` before performing any authorization checks.

It is safer to scope the `signed material` when calculating the leafs as well. Including the current Midnight address in the signed payload or domain separator would make orders instance-specific even if a ratifier is mistakenly authorized elsewhere.

This should be covered with a regression test that deploys two `Midnight` contracts and verifies that a signer authorized only on one instance cannot ratify orders on the other.

Morpho: Fixed in [PR 744](#).

Spearbit: Fix verified.

5.2.7 `ApprovalRatifier` approvals can replay across Midnight deployments

Severity: Low Risk.

Context: `ApprovalRatifier.sol#L9-L23`.

Description: `ApprovalRatifier` (`ApprovalRatifier.sol#L12-L20`) stores approvals only as `approved[maker][root]`. There is no scoping by Midnight instance, and `Midnight.take` (`Midnight.sol#L270-L271`) accepts any ratifier that the maker authorized in the current deployment.

Because `Offer` (`IMidnight.sol#L21-L38`) does not include the current Midnight address, the same approved root can be replayed across multiple Midnight contracts on the same chain when they share the same `ApprovalRatifier`. This means a maker may believe they approved an order set for one deployment while that approval remains valid anywhere else that reuses the ratifier and root: x .

$$M_A \leftarrow R \rightarrow M_B$$

Recommendation: Approvals should be scoped to the Midnight instance that consumes them. The simplest fix is to require `msg.sender` to be the intended Midnight and to key approvals by `(msg.sender, maker, root)` instead of only `(maker, root)`.

If deployment simplicity matters more than ratifier reuse, an acceptable alternative is to deploy one `ApprovalRatifier` per Midnight instance and treat ratifiers as strictly single-instance components.

This should also be covered with a regression test that deploys two Midnight contracts sharing one `ApprovalRatifier` and verifies that an approval created for one deployment cannot be consumed by the other.

Morpho: Fixed in [PR 744](#).

Spearbit: The [PR 744](#) fixes this finding. The contract has been renamed to `SetterRatifier`. Moreover additional authority has been added to authorised users of a `maker` on the pinned `Midnight` instance for this ratifier to approve `root` s.

5.2.8 Calls to `take()` can be forced to revert via front-running

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: In `take()`, offers can only be filled to their specified maximum limits:

```
uint256 newConsumed;
if (offer.maxSellerAssets > 0) {
    newConsumed = consumed[offer.maker][offer.group] += sellerAssets;
    require(newConsumed <= offer.maxSellerAssets, "consumed seller assets");
} else if (offer.maxBuyerAssets > 0) {
    newConsumed = consumed[offer.maker][offer.group] += buyerAssets;
    require(newConsumed <= offer.maxBuyerAssets, "consumed buyer assets");
} else {
    newConsumed = consumed[offer.maker][offer.group] += units;
    require(newConsumed <= offer.maxUnits, "consumed units");
}
```

An attacker can force a call to `take()` which fills an offer entirely (ie. consumes the maker's entire max limit) to revert by front-running it and calling `take()` with dust amounts, causing the `newConsumed<=maxUnits` check to revert in the subsequent `take()` call.

Note that in general, even not considering these intentional dust attacks, there could be to in-flight (mempool) transactions that could cause the other one to revert.

Recommendation: A possible mitigation would be to add logic which dynamically calculates the amount of units needed to fill an offer entirely based on `maxUnits-consumed`. This can be done in a separate periphery contract.

Alternatively, to make it less economically feasible for the attacker one can introduce dynamic lower bounds for units. The parameters for this dynamically calculated units lower bound can be included in the obligation structure. This should also make it less enticing for attackers to lend loans in dust amounts.

Morpho: We acknowledge this. This is a known behavior and is handled by the bundler: [MidnightBundles.sol](#).

Spearbit: Acknowledged.

5.2.9 Continuous fee calculation could round down to zero for tokens with small decimals

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: In `updatePositionView()`, the continuous fee is calculated based on the pending fee multiplied by a percentage of the time passed relative to the time to maturity:

```
uint128 fee = _lastAccrual < obligation.maturity
    ? uint128(postSlashPending.mulDivDown(accrualEnd - _lastAccrual, obligation.maturity -
        _lastAccrual))
    : 0;
```

For extremely small decimal tokens (eg. 4 decimals or less), this could round down to zero if the time passed since last accrual (ie. `accrualEnd-_lastAccrual`) is small relative to the time remaining to maturity (ie. `obligation.maturity-_lastAccrual`).

Fortunately, since both the numerator and denominator become smaller when `_lastAccrual` is updated, the rounding error becomes less and the full pending fee will be paid out eventually. Therefore, this is only a concern if the lender withdraws his entire credit way ahead of time before an obligation's maturity.

Recommendation: Since this isn't an issue for tokens with 6 decimals or more, consider adding under the token requirements that tokens with decimals less than 6 should not be used.

Morpho: This was commented already:

```
/// @dev Because of roundings, trading and continuous fees might charge less than expected,  
→ which can become problematic  
/// for chains where the gas is cheaper than 1 asset of the loan token.
```

(thus it's not really an issue).

We don't think that it should be added to the token requirements, as only the fees break.

Spearbit: Acknowledged.

5.2.10 Obligation accounting breaks in the event of a hardfork

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: The ID of an obligation includes the current chain ID:

```
function toId(Obligation memory obligation) public view returns (bytes32) {  
    return IdLib.toId(obligation, block.chainid, address(this));  
}
```

However, if a hard fork occurs and `block.chainid` changes, the ID of all obligations will also change which breaks all accounting.

Recommendation: Since the likelihood of this occurring is quite low, consider documenting this risk.

Morpho: Fixed in [PR 738](#).

Spearbit: Fix verified.

5.3 Gas Optimization

5.3.1 Verification and potential optimisation of `countBits` implementation

Severity: Gas Optimization.

Context: [UtilsLib.sol#L69-L76](#), [Midnight.sol#L458-L462](#).

Description:

```
function countBits(uint128 x /* x0 */) internal pure returns (uint256) {  
    unchecked {  
        // x1  
        x = x - ((x >> 1) & 0x55555555555555555555555555555555);  
  
        // x2  
        x = (x & 0x33333333333333333333333333333333) + ((x >> 2) &  
            → 0x33333333333333333333333333333333);  
  
        // x3  
        x = (x + (x >> 4)) & 0x0f0f0f0f0f0f0f0f0f0f0f0f0f0f0f;  
    }
```

```

    return (
        x * 0x01010101010101010101010101010101 // x4 and x5 (implicit trick due to
            ↪ `uint128` type and `unchecked`)
        ) >> 120 // x6;
    }
}

```

Assume x has the following binary representation:

$$\begin{aligned}
 x &= (b_{127} \cdots b_1 b_0)_2 &= \sum_{i=0}^{127} b_i 2^i \\
 a_0 &= 0x5 \cdots 55 = (\underbrace{01 \cdots 01}_{128})_2 &= \sum_{i=0}^{63} (01)_2 2^{2i} \\
 a_1 &= 0x3 \cdots 33 = (\underbrace{0011 \cdots 0011}_{128})_2 &= \sum_{i=0}^{31} (0011)_2 2^{4i} \\
 a_2 &= 0x0f \cdots 0f0f = (\underbrace{00001111 \cdots 00001111}_{128})_2 &= \sum_{i=0}^{15} (00001111)_2 2^{8i} \\
 a_3 &= 0x01 \cdots 0101 = (\underbrace{00000001 \cdots 00000001}_{128})_2 &= \sum_{i=0}^{15} 2^{8i}
 \end{aligned}$$

Let.

$$\begin{aligned}
 x &= x_0 \leftarrow \\
 x_1 &\leftarrow x_0 - ((x_0 \gg 1) \wedge a_0) \\
 x_2 &\leftarrow (x_1 \wedge a_1) + ((x_1 \gg 2) \wedge a_1) \\
 x_3 &\leftarrow (x_2 + (x_2 \gg 4)) \wedge a_2 \\
 x_4 &\leftarrow x_3 \cdot a_3 \\
 \textcolor{red}{x_5} &\leftarrow x_4 \wedge (2^{128} - 1) \\
 x_6 &\leftarrow x_5 \gg 120
 \end{aligned}$$

Note: The x_5 calculation is implicit due to the type of `x` being `uint128` and operations being performed in an `unchecked` block:

```

// wrapped multiplication which actually truncates/masks the result
x * 0x01010101010101010101010101010101

```

This is actually really important, perhaps one should leave a comment to highlight this.

The first 3 lines of `countBits` transform x as follows:

$$\begin{aligned}
 x = x_0 &\leftarrow \sum_{i=0}^{127} b_i 2^i &= \cdots b_{15} b_{14} b_{13} b_{12} b_{11} b_{10} b_9 b_8 \quad b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0 \\
 x_1 &\leftarrow \sum_{i=0}^{63} (b_{2i+1} + b_{2i+0}) 2^{2i} &= \cdots 0(b_{15} + b_{14}) 0(b_{13} + b_{12}) 0(b_{11} + b_{10}) 0(b_9 + b_8)
 \end{aligned}$$

$$\begin{aligned}
x_2 &\leftarrow \sum_{i=0}^{31} (b_{4i+3} + b_{4i+2} + b_{4i+1} + b_{4i+0}) 2^{4i} &= \dots & 0(b_7 + b_6)0(b_5 + b_4)0(b_3 + b_2)0(b_1 + b_0) \\
& & & 000(b_{15} + b_{14} + b_{13} + b_{12})000(b_{11} + b_{10} + b_9 + b_8) \\
& & & 000(b_7 + b_6 + b_5 + b_4)000(b_3 + b_2 + b_1 + b_0) \\
x_3 &\leftarrow \sum_{i=0}^{15} \left(\begin{array}{c} b_{8i+7} + b_{8i+6} + b_{8i+5} + b_{8i+4} \\ + b_{8i+3} + b_{8i+2} + b_{8i+1} + b_{8i+0} \end{array} \right) 2^{8i} &= \dots & 0000000(b_{15} + b_{14} + b_{13} + b_{12} + b_{11} + b_{10} + b_9 + b_8) \\
& & & 0000000(b_7 + b_6 + b_5 + b_4 + b_3 + b_2 + b_1 + b_0) \\
x_4 &\leftarrow \sum_{i=0}^{30} \left(\sum_{\substack{j+k=i \\ 0 \leq j \leq 15 \\ 0 \leq k \leq 15}} (b_{8j+7} + \dots + b_{8j+0}) \right) 2^{8i} &= \dots & \\
x_5 &\leftarrow \sum_{i=0}^{15} \left(\sum_{\substack{j+k=i \\ 0 \leq j \leq 15 \\ 0 \leq k \leq 15}} (b_{8j+7} + \dots + b_{8j+0}) \right) 2^{8i} &= \dots & \\
x_6 &\leftarrow \sum_{\substack{j+k=15 \\ 0 \leq j \leq 15 \\ 0 \leq k \leq 15}} (b_{8j+7} + \dots + b_{8j+0}) = \sum_{i=0}^{127} b_i
\end{aligned}$$

Important facts:

- Addition of bit counts within groups do not spill/carry into the next group:

$$\sum_{j=0}^{2^i-1} b_{2^i k+j} < 2^i$$

Recommendation: One can shave off a few gas from the current implementation since one knows that at the current call sites the provided value has at most 11 set bits. The following implementation works for uint128 numbers with up to 15 bits:

```
function countBitsLe15(uint128 x) internal pure returns (uint256) {
    unchecked {
        x = x - ((x >> 1) & 0x55555555555555555555555555555555);
        x = (x & 0x33333333333333333333333333333333) + ((x >> 2) &
            ↪ 0x33333333333333333333333333333333);
        return (x * 0x11111111111111111111111111111111) >> 124;
    }
}
```

One avoid summing the bits in the 8-bit lanes. One can make it even simpler by using only 3 operations, if one restricts to total of 48 collateral per obligation per which only 30 can have activated bits:

```
function countBitsLe30(uint48 x) internal pure returns (uint256 res) {
    assembly ('memory-safe') {
        // If desired to cleanup potentially dirty upper bits:
        // x := and(x, 0xfffffffffff)
        res := mod(
            and(
```



```

mul(
    x, // if needed also cleanup upper bits of x
    0x1000000000001000000000001000000000001000000000001
),
    0x8421084210842108421084210842108421084210842108421084210842108421
),
    0x1f
)
}
}

```

The idea for the above comes from:

$$\sum_{j=0}^N b_j = \left(\left(\sum_{j=0}^N b_j 2^j \right) \left(\sum_{j=0}^{q-1} 2^{dj} \right) \right) \wedge \left(\sum_{j=0}^{\left\lfloor \frac{N+d(q-1)}{q} \right\rfloor} 2^{qj} \right) \pmod{2^q - 1}$$

where.

$$\begin{aligned} \gcd(d, q) &= 1 \\ N &< d \\ N + d(q - 1) &< 256 \quad (\text{required in EVM}) \end{aligned}$$

Also it is best to apply the following to get more out of these functions (plus optimisation for bad paths to avoid storage updates):

```

diff --git a/src/Midnight.sol b/src/Midnight.sol
index 018f8904..216384ec 100644
- -- a/src/Midnight.sol
+ ++ b/src/Midnight.sol
@@ -456,9 +456,10 @@ contract Midnight is IMidnight {
    _position.collateral[collateralIndex] = UtilsLib.toUint128(oldCollateral + assets);

    if (oldCollateral == 0 && assets > 0) {
-         uint128 newBitmap = _position.activatedCollaterals.setBit(collateralIndex);
+         uint256 currentBitmap = _position.activatedCollaterals;
+         require(UtilsLib.countBits(currentBitmap) < MAX_COLLATERALS_PER_BORROWER, "too
+ many activated collaterals");
+         uint128 newBitmap = currentBitmap.setBit(collateralIndex);
        _position.activatedCollaterals = newBitmap;
-         require(UtilsLib.countBits(newBitmap) <= MAX_COLLATERALS_PER_BORROWER, "too many
- activated collaterals");
    }

    emit EventsLib.SupplyCollateral(msg.sender, id, collateralToken, assets, onBehalf);

```

The suggested diff requires the implementation of `countBits(...)` to be able provide the correct value for numbers that have at most `MAX_COLLATERALS_PER_BORROWER` bits set. Whereas the current implementation requires the function to work correctly at least on values up to `MAX_COLLATERALS_PER_BORROWER+1` bits set.

Morpho: We acknowledged the finding.

Spearbit: Acknowledged.

5.4 Informational

5.4.1 Minor issues (typos, comments, ...)

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description/Recommendation:

1. [Midnight.sol#L469](#):

```
- ... if all the collateral is withdrawn and the borrower has no debt.  
+ ... if the borrower has no debt.
```

2. [Midnight.sol#L643-L648](#): `setIsAuthorized` being transitive might not be desired from the `user` perspective (ie an authorised `msg.sender` on behalf of a user being able to authorise another account).
3. [Midnight.sol#L455-L462](#), [Midnight.sol#L482-L487](#), [Midnight.sol#L595-L599](#): Refactor position collateral amount and `activatedCollaterals` updates into utility functions to make sure the required updates happen atomically to avoid potential future mistakes.
4. [UtilsLib.sol#L9](#): Cheaper to perform below, but it's also ok to keep the current implementation to align with the overloaded `atMostOneNonZero` with three inputs:

```
diff --git a/src/libraries/UtilsLib.sol b/src/libraries/UtilsLib.sol  
index 7e3826ea..2282ecd2 100644  
- -- a/src/libraries/UtilsLib.sol  
+ ++ b/src/libraries/UtilsLib.sol  
@@ -6,7 +6,7 @@ library UtilsLib {  
    /// @dev Returns true if at most one of `x` and `y` is nonzero.  
    function atMostOneNonZero(uint256 x, uint256 y) internal pure returns (bool z) {  
        assembly {  
-            z := gt(add(iszero(x), iszero(y)), 0)  
+            z := iszero(mul(x, y))  
        }  
    }  
}
```

5. [TickLib.sol#L5](#): Improve comment to `floor(ln(1+0.025)*1e18)`.
6. [TickLib.sol#L24](#): Add comment to show the explicit formula used to derive this constant `floor(ln(2)*1e18)`.
7. [TickLib.sol#L6](#): Make sure all relevant documents (Notion, ...) are updated to indicate this value for max tick `1024`.
8. [Midnight.sol#L271](#), [Midnight.sol#L384](#), [Midnight.sol#L397](#), [Midnight.sol#L440](#), [Midnight.sol#L619](#), [Midnight.sol#L653](#), [ConstantsLib.sol#L15](#): Make sure the `onCallbackFuncs` (`onRatify`, `onBuy`, `onSell`, `onRepay`, `onLiquidate`, `onFlashLoan`) return name spaced scoped per endpoint magic bytes instead of either no check or one magic sequence shared among all callbacks. One can use a following pattern:

```
keccak256("morpho-labs/midnight/{onCallbackFuncs}/success")
```

9. [Midnight.sol#L270](#): Define a separate access control storage parameter and logic for allow ratifiers (separate from authorised users which are allowed as callers to specific endpoints).
10. [EcrecoverRatifier.sol#L24](#): Define a separate access control for users that can sign `root` digests on behalf of someone else this storage parameter can be added in `EcrecoverRatifier`.
11. [Midnight.sol#L261-L264](#), [Midnight.sol#L410-L412](#), [Midnight.sol#L431-L432](#), [Midnight.sol#L450-L452](#), [Midnight.sol#L477-L479](#): Inconsistent order of the following checks:

```
require(onBehalf == msg.sender || isAuthorized[onBehalf][msg.sender], "unauthorized");
bytes32 id = touchObligation(obligation);
```

make sure all required endpoints first check authorisation of the `msg.sender` then touch the obligation. Finally perform other required operations.

Spearbit:

verified	item	notes
✓	1	Fixed in PR 736
✓	2	There is already a comment about that. do you think that we should do something further? ...
✓	3	Acknowledged
✓	4	Fixed in PR 736
✓	5	Fixed in PR 736
✓	6	Fixed in PR 736
out-of-scope	7.	Fixed in notion
✓	8	Partially fixed in PR 786 by namespacing <code>morpho</code> into the hashed string. Different call-back endpoints still send clashing magic values.
✓	9	Acknowledged
✓	10	Acknowledged
✓	11	Fixed in PR 736

5.4.2 A phantom collateral can block liquidations

Severity: Informational.

Context: [Midnight.sol#L531](#).

Description: Assume an obligation of a loan token can be backed by many collaterals. All except one are real trusted sources of collateral. The one collateral is only there to be used in the future to block collateral.

A user can back its loan by mix of all collateral and maybe only one small unit of the phantom collateral. Then if the position had debt and matured or if it is unhealthy (which could possibly be blocked from checking) the phantom's collateral's oracle could `revert` on `price()` fetches and thus block the whole liquidation flow. It can also exaggerate the price of the phantom oracle to let the user withdraw the other supplied collaterals to back the position.

Recommendation: It would be the responsibility of the lenders to make sure they check the legitimacy of all the provided collaterals in an obligation since a barely used phantom collateral can later lock the funds to be returned.

There is already a NatSpec clause in [Midnight.sol#L86-L87](#) regarding the liveness of the price oracle:

```
/// @dev If an activated collateral oracle reverts on `price`, `liquidate`, `isHealthy`,
↪ `withdrawCollateral` when the
/// borrower has debt, and `take` whenever the seller still has debt all revert.
```

But perhaps the above scenario can be highlighted further.

Morpho: This scenario is now documented in [PR 767](#).

```
/// A single reverting oracle blocks liquidation for every borrower with that collateral
↳ activated, and a borrower can
/// activate such a collateral post-incident to block their own liquidation.
```

Spearbit: Fix verified.

5.4.3 In the presence of liquidity the lenders and the `feeClaimer` can avoid their credits getting slashed

Severity: Informational.

Context: [Midnight.sol#L547-L563](#).

Description: If an obligation had enough loan token liquidity that can be withdrawn users and `feeClaimer` can frontrun calls to liquidation to avoid their credits getting slashed. This can create a race condition for these entities to try to capture their credits while supply of loan tokens last. An extreme case would be when slashing is about to happen and `obligationState[id].withdrawable` is `0`.

Recommendation: The above can be documented and analysed.

Morpho: Fixed in [PR 872](#).

Spearbit: Fix verified.

5.4.4 `clz` opcode is not supported on all almost EVM-compatible chains

Severity: Informational.

Context: [UtilsLib.sol#L78-L83](#).

Description: List of supported chain for Morpho v2 products (vaults, ...) are listed in <https://docs.morpho.org/get-started/resources/addresses>. Based this list:

Chain	CLZ support	Probe result
Ethereum	Yes	Returned <code>0x...00ff</code> for <code>CLZ(1)</code> and <code>0x...0100</code> for <code>CLZ(0)</code>
Arbitrum	Yes	Returned <code>0x...00ff</code> and <code>0x...0100</code>
Avalanche	No	<code>invalidopcode:opcode0xlenotdefined</code>
Base	No	<code>invalidopcode:CLZ</code>
Botanix	No	<code>EVMError:OpcodeNotFound</code>
Citrea	No	<code>EVMError:OpcodeNotFound</code>
Cronos	No	<code>invalidopcode:opcode0xlenotdefined</code>
Eden	Yes	Returned <code>0x...00ff</code> and <code>0x...0100</code>
Gensyn	No	<code>invalidopcode:CLZ</code>
HyperEVM	No	<code>EVMError:OpcodeNotFound</code>
Katana	No	<code>invalidopcode:CLZ</code>
Linea	Yes	Returned <code>0x...00ff</code> and <code>0x...0100</code>
Monad	Yes	Returned <code>0x...00ff</code> and <code>0x...0100</code>

OP Mainnet	No	EVMError:NotActivated
Pharos	No	Anundefinedinstructionhasbeenencountered
Plasma	No	EVMError:NotActivated
Plume	Yes	Returned 0x...00ff and 0x...0100
PolygonPOS	Yes	Returned 0x...00ff and 0x...0100
Stable	No	invalidopcode:CLZ
Tempo	Yes	Returned 0x...00ff and 0x...0100
Unichain	No	invalidopcode:CLZ
World Chain	No	EVMError:NotActivated

Notes:

- Pharos was checked via its publicly documented **Atlantic Testnet** RPC, not a listed mainnet RPC.
- Tempo was checked via its publicly listed **Moderato testnet** RPC, since that is what is publicly available.
- The positive cases were sanity-checked with both CLZ(1)=255 and CLZ(0)=256, not just a single opcode acceptance test.

The chains that don't have a support currently:

Chain	Public plan to add CLZ ?	Hardfork / upgrade	Date
Base	Yes	Base V1	Mainnet: Early May 2026, TBD. Sepolia: April 20, 2026 18:00 UTC
OP Mainnet	Likely, but not explicitly confirmed in public docs	Karst	No public timestamp found
Avalanche	No public plan found	—	—
Botanix	No public plan found	—	—
Citrea	No public plan found	—	—
Cronos	No public plan found	—	—
Gensyn	No public plan found	—	—
HyperEVM	No public plan found	—	—
Katana	No public plan found	—	—
Pharos	No public plan found	—	—
Plasma	No public plan found	—	—
Stable	No public plan found	—	—
Unichain	No public chain-specific plan found	—	—
World Chain	No public chain-specific plan found	—	—

Recommendation: If Morpho would want to deploy Midnight to a wider set of chains sooner, it would be best to use a different implementation of msb that avoids the usage of the newly introduced CLZ opcode.

List of other opcodes to potentially check (although most of the above seem to support):

- TLOAD .
- TSTORE .
- MCOPY .
- PUSH0 .

Morpho: Fixed in [PR 737](#).

Spearbit: Documentation has been added in [PR 737](#).

5.4.5 Trust assumptions for `buyerCallback`

Severity: Informational.

Context: [Midnight.sol#L384](#).

Summary:

Finding Description: The Midnight only calls the `buyerCallback` with the computed buyer / seller, and then uses the buyer callback as the payer in the transfer path. It is therefore the callback contract's responsibility to verify that the buyer/seller is the expected one, and that the ID, obligation, buyerAssets / sellerAssets, units, and callback data match the flow it intends to support. Otherwise, a maker-funded callback can end up paying a trade the maker did not intend.

Example:

- A maker deploys a funded callback contract to support buy offers.
- Its `onBuy()` only checks `msg.sender==Midnight` and blindly approves `buyerAssets`.
- Later, the maker posts a sell offer.
- A malicious taker takes that sell offer and sets `takerCallback` to the maker's funded callback.
- In the sell path, Midnight treats `takerCallback` as the `buyerCallback`, calls `onBuy(...,buyer=malicious_taker,...)`, and then pulls funds from that callback as the payer.
- If the callback does not verify `buyer==maker` and does not distinguish trusted maker-side data from taker-controlled data, it can end up paying for the attacker's purchase.

Recommendation: Document explicitly that callback contracts are responsible for authenticating the full trade context before approving any transfer.

Morpho: We acknowledge this issue. We think that the behavior is pretty clear and safe already.

Spearbit: Acknowledged.

5.4.6 Loss index inflation

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Description: Under certain edge-case market conditions, inflating the obligation loss index i_O may be materially easier than in typical states.

In particular, if `totalUnits` is small and the minimum viable loan and collateral amounts are both low, an attacker may be able to repeatedly open dust positions, avoid repayment, and later self-liquidate those positions to realize bad debt. Because each bad-debt event increases i_O , this allows the attacker to inflate the loss index over time at relatively low cost.

This attack becomes easier, for example, when:

- The collateral-to-loan price remains stable; and.
- The collateral `lltv` is equal, or very close, to `WAD` .

To end up in a state where `lossIndex` i_O maxes out, there could be 843 bad debt liquidations at `10%` of the total units.

An attacker could then execute the following sequence:

- Inflate i_O to near its maximum value;
- Allow honest users to open positions while the loss index is already heavily inflated; and.
- Perform additional bad-debt liquidations to fully saturate i_O at $2^{128} - 1$.

In other words, honest users' position creation can be sandwiched between two loss-index inflation phases, causing them to enter after most of the inflation has already occurred but before final saturation.

Recommendation: These edge-case scenarios should be investigated further, especially under low- `totalUnits` conditions and near- `WAD` `lltv` configurations. If this behavior is confirmed, the protocol should consider protocol-level mitigations rather than relying on users to avoid opening positions when i_O is already highly inflated, since the inflation itself may be front-runnable.

Morpho: We acknowledge this issue. Note that the core issue (rounding on `lossIndex`) was documented already.

Spearbit: Acknowledged.

6 Appendix

6.1 Formulas and invariants

Under ideal conditions, one would expect that:

$$a_w^{\mathcal{O}} + \underbrace{\sum_u D_{\mathcal{O},u}}_{\text{total obligation debt}} = a_{tot}^{\mathcal{O}} = a_{\mathcal{O}}^{credit} + \sum_u a_{\mathcal{O},u}^{credit}$$

But due to lazy slashing and rounding errors above is not true in general (need to be further analysed). Instead if all positions were updated so the position credits would not hold stale values we would get:

$$a_w^{\mathcal{O}} + \underbrace{\sum_u D_{\mathcal{O},u}}_{\text{total obligation debt}} = a_{tot}^{\mathcal{O}} \geq a_{\mathcal{O}}^{credit} + \sum_u a_{\mathcal{O},u}^{credit}$$

6.1.1 Credit/Debt

If one would instead of two parameters of credit and debt were to work with `signed` integers, we could define:

$$a_{\mathcal{O},u}^* = a_{\mathcal{O},u} - D_{\mathcal{O},u}$$

and then one could show that:

$$\begin{aligned} a_{\mathcal{O},u} &= + [a_{\mathcal{O},u}^*]^+ \\ D_{\mathcal{O},u} &= - [a_{\mathcal{O},u}^*]^- \end{aligned}$$

and that: $(a_{\mathcal{O},u}, D_{\mathcal{O},u}) \in \mathbb{Z}_{\geq 0}^2$

Loss Index: Let $i_{\mathcal{O}}$ be the loss index for the obligation $\mathcal{O}$. Define:

$$i_{\mathcal{O}}^- = (2^{128} - 1) - i_{\mathcal{O}}$$

Then we have:

$$i_{\mathcal{O}}^- = \left\lfloor \frac{a_{tot}^{\mathcal{O}} - D_{\mathcal{O},u}^{bad}}{a_{tot}^{\mathcal{O}}} \underbrace{[\dots [2^{128} - 1]]}_{\text{old loss index bar}} \right\rfloor$$

in an ideal world one would just keep track of the exact value:

$$\prod_{E_{liq}} \frac{a_{tot}^{\mathcal{O}} - D_{\mathcal{O},u}^{bad}}{a_{tot}^{\mathcal{O}}}$$

6.1.2 Some invariants

Past maturity the amount of total debt per obligation cannot grow $\sum_u D_{\mathcal{O},u}$.

6.1.3 Notation

parameter	description
$\mathcal{O}$	A specific obligation. Used for indexing. It could be the obligation <code>id</code> or the obligation as a whole depending on the context
$a_w^{\mathcal{O}}$	<code>obligationState[id].withdrawable</code>
$D_{\mathcal{O},u}$	<code>position[id][u].debt</code>
$a_{tot}^{\mathcal{O}}$	<code>obligationState[id].totalUnits</code>
$a_{\mathcal{O}}^{credit}$	<code>obligationState[id].continuousFeeCredit</code>
$a_{\mathcal{O},u}^{credit}$	<code>position[id][u].credit</code>
$i_{\mathcal{O}}$	<code>obligationState[id].lossIndex</code>
$D_{\mathcal{O},u}^{bad}$	<code>badDebt</code>
E_{liq}	Set of liquidation events

6.2 Debt Accounting + Liquidation

Context: [Midnight.sol#L534-L537](#), [Midnight.sol#L566-L571](#), [Midnight.sol#L579-L593](#)

In the liquidation flow bad debt of a position is calculated as:

$$\begin{aligned}
D_{\mathcal{O},u}^{max} &= \sum_{T \in \mathcal{O}} \left[\left\lfloor c_{\mathcal{O},u}^T \cdot \frac{p_{\mathcal{O},T}}{10^{18+18}} \right\rfloor \frac{\text{lltv}_{\mathcal{O},T}}{10^{18}} \right] \\
D_{\mathcal{O},u}^{dis}(\text{lif}) &= \sum_{T \in \mathcal{O}} \left[\left\lfloor c_{\mathcal{O},u}^T \cdot \frac{p_{\mathcal{O},T}}{10^{18+18}} \right\rfloor \frac{10^{18}}{\text{lif}_T} \right] \\
D_{\mathcal{O},u}^{bad} &= \left[D_{\mathcal{O},u} - D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},T}^{max}) \right]^+
\end{aligned}$$

(in the above $[x]^+ = \max(0, x)$).

With the current choice of `lifo,tmax` and `lltvo,t` we have:

LLTV: 0.385	LIF: 1.181683899556868537	LIF x LLTV: 0.454948301329394387
LLTV: 0.385	LIF: 1.444043321299638989	LIF x LLTV: 0.555956678700361011
LLTV: 0.625	LIF: 1.103448275862068965	LIF x LLTV: 0.689655172413793104
LLTV: 0.625	LIF: 1.230769230769230769	LIF x LLTV: 0.769230769230769231
LLTV: 0.77	LIF: 1.061007957559681697	LIF x LLTV: 0.816976127320954907
LLTV: 0.77	LIF: 1.129943502824858757	LIF x LLTV: 0.870056497175141243
LLTV: 0.86	LIF: 1.036269430051813471	LIF x LLTV: 0.891191709844559586
LLTV: 0.86	LIF: 1.075268817204301075	LIF x LLTV: 0.924731182795698925
LLTV: 0.915	LIF: 1.021711366538952745	LIF x LLTV: 0.934865900383141762
LLTV: 0.915	LIF: 1.044386422976501305	LIF x LLTV: 0.955613577023498695
LLTV: 0.945	LIF: 1.01394169835234474	LIF x LLTV: 0.95817490494296578
LLTV: 0.945	LIF: 1.028277634961439588	LIF x LLTV: 0.971722365038560411
LLTV: 0.965	LIF: 1.008827238335435056	LIF x LLTV: 0.97351828499369483
LLTV: 0.965	LIF: 1.017811704834605597	LIF x LLTV: 0.982188295165394402
LLTV: 0.98	LIF: 1.005025125628140703	LIF x LLTV: 0.984924623115577889
LLTV: 0.98	LIF: 1.010101010101010101	LIF x LLTV: 0.989898989898989899
LLTV: 1	LIF: 1	LIF x LLTV: 1
LLTV: 1	LIF: 1	LIF x LLTV: 1

$$\frac{\text{lif}_{\mathcal{O},T}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T}}{10^{18}} \leq 1$$

or

$$\frac{\text{lltv}_{\mathcal{O},T}}{10^{18}} \leq \frac{10^{18}}{\text{lif}_{\mathcal{O},T}^{max}} \leq \frac{10^{18}}{10^{18}} = 1$$

So for

$$\text{lif}_T \in [10^{18}, \text{lif}_{\mathcal{O},T}^{max}]$$

we have:

$$D_{\mathcal{O},u}^{max} \leq D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},-}^{max}) \leq D_{\mathcal{O},u}^{dis}(\text{lif})$$

so

$$D_{\mathcal{O},u} \rightarrow \begin{cases} \text{healthy} & \in [0, D_{\mathcal{O},u}^{max}] \\ \text{unhealthy but no bad debt} & \in (D_{\mathcal{O},u}^{max}, D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},-}^{max})] \\ \text{unhealthy but with bad debt} & \in [D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},-}^{max}), 2^{128}] \end{cases}$$

$$a_{repaid}^{max} = \begin{cases} \left\lceil \left(\min(D_{\mathcal{O},u}, D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},T}^{max})) - D_{\mathcal{O},u}^{max} \right) \left[\frac{10^{18}}{10^{18} - \text{lif}_{\mathcal{O},T_j}^{max} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}} \right] \right\rceil & \text{lltv}_{\mathcal{O},T_j} < 10^{18} \\ 2^{256} - 1 & \text{(no upper bound)} \end{cases}$$

in general one can prove:

$$\begin{aligned} a_{repaid} &\leq \left\lceil \left[c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rceil & a_j > 0 \\ a_{repaid} &\leq \left\lceil \left[(c_{\mathcal{O},u}^{T_j} + 1) \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rceil & a_j = 0 \end{aligned}$$

we have:

$$\begin{aligned} \left\lceil \left[c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rceil &\leq \left\lceil \left[c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rceil + 1 + 1 \\ \left\lceil \left[(c_{\mathcal{O},u}^{T_j} + 1) \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rceil &\leq \left\lceil \left[c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rceil + 1 + \left\lceil \left[-\epsilon + \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rceil \end{aligned}$$

where

$$\epsilon = 1 - \left\{ c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right\} \in [0, 1)$$

The recovery close factor condition ([Midnight.sol#L580-L592](#)) below is not quite using the true upperbound for a_{repaid} ($\tau_{\mathcal{O}}$ is the `rcfThreshold`):

$$a_{repaid} \leq a_{repaid}^{max} \quad \text{or} \quad \left\lfloor \left[c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rfloor < a_{repaid}^{max} + \tau_{\mathcal{O}}$$

One can rewrite the 2nd condition as:

$$a_{repaid} \leq \left\lfloor \left[c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rfloor + 1 + F \left(1, \left\lfloor \left[-\epsilon + \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rfloor \right) < a_{repaid}^{max} + \tau_{\mathcal{O}} + 1 + F \left(1, \left\lfloor \left[-\epsilon + \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rfloor \right)$$

where: $F(x, y) = \begin{cases} x & a_j > 0 \\ y & a_j = 0 \end{cases}$

i.e.

$$a_{repaid} < a_{repaid}^{max} + \tau_{\mathcal{O}} + 1 + F \left(1, \left\lfloor \left[-\epsilon + \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_{\mathcal{O},T_j}^{max}} \right\rfloor \right)$$

6.3 Risky collaterals

$$\text{lif}_{\mathcal{O},T_j}^{max} = \text{lltv}_{\mathcal{O},T_j} = 10^{18}$$

Very risky obligations when all collaterals have risky liquidation parameters

$$D_{\mathcal{O},u}^{dis}((10^{18}, \dots)) = D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},-}^{max}) \approx D_{\mathcal{O},u}^{max}$$

$$\text{error} = D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},-}^{max}) - D_{\mathcal{O},u}^{max} \in [0, 10]$$

6.3.1 Notations

parameter	description
$\mathcal{O}$	A specific obligation. Used for indexing. It could be the obligation <code>id</code> or the obligation as a whole depending on the context

u	A specific user/position owner
T	A collateral token in the obligation collateral set
T_j	The collateral token seized during liquidation
$c_{\mathcal{O},u}^T$	Amount of collateral token T posted by user u in obligation $\mathcal{O}$
$p_{\mathcal{O},T}$	Oracle price for collateral token T in obligation $\mathcal{O}$, scaled by 10^{36} in the formulas above
$D_{\mathcal{O},u}$	<code>position[id][u].debt</code>
$D_{\mathcal{O},u}^{max}$	Maximum healthy debt of position $(\mathcal{O}, u)$ under the obligation LLTVs
$D_{\mathcal{O},u}^{dis}(\text{lif})$	Discounted collateral value of position $(\mathcal{O}, u)$ under liquidation incentive factors lif
$D_{\mathcal{O},u}^{bad}$	<code>badDebt</code>
$D_{\mathcal{O},u}^{capped}$	Debt considered by the recovery close factor calculation, capped at $D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},T}^{max})$
$\text{lltv}_{\mathcal{O},T}$	Loan-to-value threshold for collateral token T in obligation $\mathcal{O}$
lif_T	Liquidation incentive factor for collateral token T
$\text{lif}_{\mathcal{O},T}^{max}$	Maximum liquidation incentive factor allowed for collateral token T in obligation $\mathcal{O}$
$\text{lif}_{\mathcal{O},-}^{max}$	Vector of maximum liquidation incentive factors across the collateral tokens in obligation $\mathcal{O}$
a_{repaid}	Amount of debt repaid by the liquidator
a_{repaid}^{max}	Maximum repayment amount allowed by the recovery close factor formula before threshold slack
a_j	Amount of seized collateral token T_j
$\tau_{\mathcal{O}}$	<code>rcfThreshold</code> for obligation $\mathcal{O}$
$\tau_{\mathcal{O},T_j}$	Token-specific upper-bound slack term for seized token T_j
ϵ	Fractional-rounding complement, defined as $1 - \left\{ c_{\mathcal{O},u}^{T_j} \cdot p_{\mathcal{O},T_j} / 10^{36} \right\}$
ϵ_3	Rounding-error term used in the less-strict recovery close factor derivation
$F(x, y)$	Branch function equal to x when $a_j > 0$ and y when $a_j = 0$
$[x]^+$	Positive part of x , equal to $\max(0, x)$